Word Embeddings (I)

CS 485, Spring 2026
Applications of Natural Language Processing

Katrin Erk
College of Information and Computer Sciences
University of Massachusetts Amherst

With slides by Brendan O'Connor

Roadmap

- **Last time**: n-gram language models
- **Coming up** later in the semester: Much more powerful language models
- Generalize over words towards multi-dimensional representations: embeddings
- Words that appear in similar contexts have similar embeddings

Roadmap

- **Today**: Embeddings
- Automatically learned word representations can be used as representations of word meaning

What is "asdfasdf"?

“ **asdfasdf**, Most Neglected American Fruit.” — NYTimes 1922

“ **asdfasdf** Recommended by U.S. Food Experts, Along With Persimmon, as War Nutrition” — NYTimes 1942

“ The **asdfasdf** is also pollinated by flies and other insects rather than by honeybees…”— NYTimes 2020

“Many people also cook with ripe **asdfasdf**, making bread, beer, ice cream, or this **asdfasdf** pudding…” — NYTimes 2020

What is a pawpaw?

I. Look it up in a dictionary

https://www.merriam-webster.com/

https://www.oed.com/

https://en.wiktionary.org/

pawpaw noun

Save Word

paw·paw

variants: *or less commonly* **papaw**

Definition of *pawpaw*

1 \ pə-ˈpȯ \ **:** PAPAYA

2 \ ˈpä-(ˌ)pȯ, ˈpȯ- \ **:** a North American tree (*Asimina triloba*) of the custard-apple family with purple flowers and an edible green-skinned fruit

also **:** its fruit

Lemma

Word Senses

Definition

pawpaw noun

Save Word

paw·paw

variants: *or less commonly* **papaw**

Definition of *pawpaw*

1 \ pə-ˈpȯ \ : PAPAYA

2 \ ˈpä-(ˌ)pȯ, ˈpȯ- \ : a North American tree (*Asimina triloba*) of the custard-apple family with purple flowers and an edible green-skinned fruit

also : its fruit

II. Look it at how its used

“ **Pawpaw**, Most Neglected American Fruit.” — NYTimes 1922

“ **Pawpaw** Recommended by U.S. Food Experts, Along With Persimmon, as War Nutrition” — NYTimes 1942

“ The **pawpaw** is also pollinated by flies and other insects rather than by honeybees…”— NYTimes 2020

“Many people also cook with ripe **pawpaws**, making bread, beer, ice cream, or this **pawpaw** pudding…” — NYTimes 2020

II. Look it at how its used

“ *Pawpaw*, Most Neglected **American Fruit** .” — NYTimes 1922

“ *Pawpaw* Recommended by U.S. Food Experts, Along With **Persimmon** , as War **Nutrition** ” — NYTimes 1942

“ The *pawpaw* is also **pollinated** by **flies** and other insects rather than by honeybees…”— NYTimes 2020

“Many people also **cook** with **ripe** *pawpaws* , making **bread** , **beer**, **ice cream** , or this *pawpaw* **pudding** …” — NYTimes 2020

Aspects of word meaning

Synonyms

- couch / sofa
- oculist / eye - doctor
- car / automobile
- water / H_2O
- draft / draught

Antonyms

- yes / no
- dark / light
- hot / cold
- up / down
- clip / clip

Aspects of word meaning

Similarity
(taxonomic similarity)

- cat / dog
- cardiologist / pulmonologist
- car / bus
- sheep / goat
- glass / mug

Relatedness
(topical similarity)

- coffee / cup
- waiter / menu
- farm / cow
- house / roof
- theater / actor

Aspects of word meaning

- Connotation: the affective meaning of a word
- Osgood (1957): Meaning as point in 3-dimensional space:
  - Valence (pleasantness)
    - unhappy, annoyed <----------> happy, satisfied
  - Arousal (intensity)
    - calm <----------> excited
  - Dominance (degree of control exerted by stimulus)
    - awed, influences <----------> controlling

Aspects of word meaning

- Connotation: the affective meaning of a word
- Osgood (1957): Meaning as point in 3-dimensional space
- Human judgments from that study:

	Valence	Arousal	Dominance
courageou	8.05	5.5	7.38
music	7.67	5.57	6.5
heartbreak	2.45	5.65	3.58
cub	6.71	3.95	4.24

Word embeddings/vectors

- We need a representation of words capable of synonyms, rough similarity, or maybe even other aspects of meaning
- Give each word a k-dimensional **vector**
  - a vector is a list of numbers
  - a vector is a point/direction in k-dimensional space

Getting at word meanings: an electronic dictionary

- How to get word meanings?
  - Lexical resources like WordNet: dictionary-like databases of word synonyms & other word-to-word relationships, constructed manually
    - WordNet in particular has been used to create data for countless NLP tasks
- see demo 1

Learning word

- Lexical resources can sometimes help, but typically don't cover all words or meanings any particular task needs

- Can we *learn* the word representations instead?

Distributional Semantics

"You shall know a word by the company it keeps!" — Firth (1957)

Intuitions: Harris (1954)

"If A and B have almost identical environments except chiefly sentences which contain both, we say they are synonyms: *oculist* and *eye- doctor* ."

Learning word

- Could we automatically *learn* word meanings?
  1. We'd like to generalize word meanings beyond individual words
  2. Information from nearby words gives information about a word: the **textual context** of a word
- What is textual context?

An old approach: Count-based distributional models

Say we have this "corpus":

They picked up red **apples** that had fallen to the ground
Eating **apples** is healthy
She ate a red **apple**
Pick an **apple**.

Now what is a context? Let's say:

- "Apple", or "apples", is our target word
- Context = three words left, three words right of the target word. Stop at sentence boundaries.
- We lemmatize all words

A small example: context word counts for “apple”

They picked up red **apples** that had fallen to the ground
Eating **apples** is healthy
She ate a red **apple**
Pick an **apple**.

3-word context, lemmatized

a	be	eat	fall	have	healthy	pick	red	that	up
2	1	2	1	1	1	2	2	1	1

A small example: context word counts for “apple”

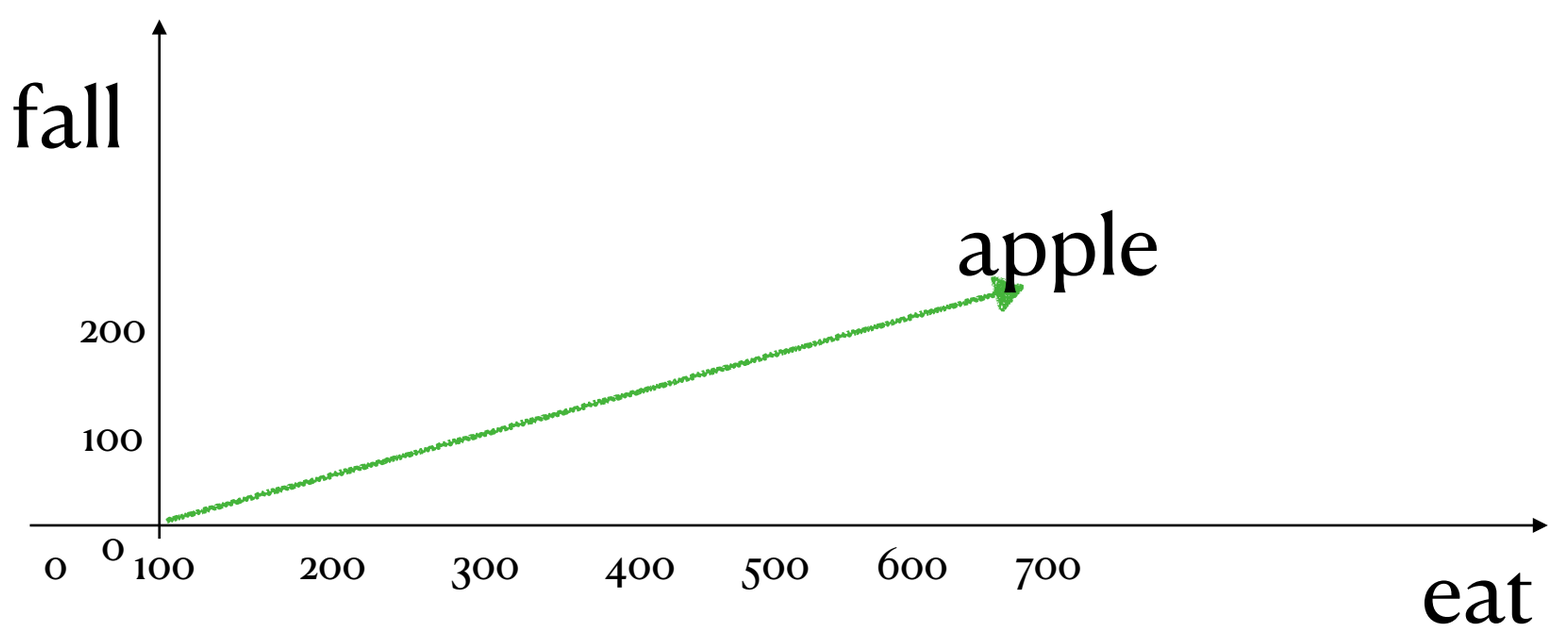
We use a bigger corpus: Context word counts for (some) context words of “apple” in the British National Corpus (100 million words)

eat	fall	ripe	slice	peel	tree	throw	fruit	pie	bite	crab
794	244	47	221	208	160	145	156	109	104	88

The core trick: View a table of counts as a set of coordinates

eat	fall	ripe	slice	peel	tree	throw	fruit	pie	bite	crab
794	244	47	221	208	160	145	156	109	104	88

Interpret counts as coordinates: First dimension is "eat" with a coordinate of 794.
Second dimension is "fall" with a coordinate of 244.



How do you measure the contextual similarity of “apple” and “orange”?

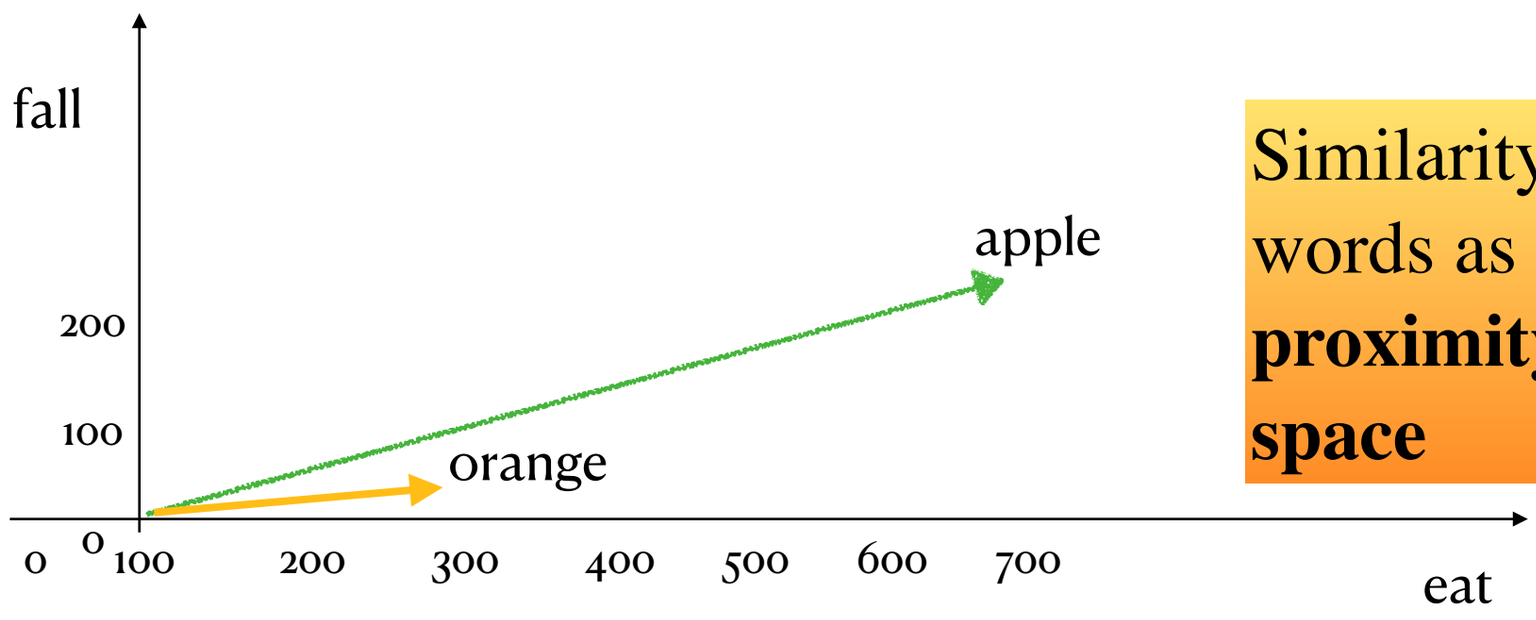
	eat	fall	ripe	slice	peel	tree	throw	fruit	pie	bite	crab
apple	794	244	47	221	208	160	145	156	109	104	88

	eat	fall	ripe	slice	peel	tree	throw	fruit	pie	bite	crab
orange	265	22	25	62	220	64	74	111	4	4	8

Count how often “apple” occurs close to other words in a large text collection (corpus). Do the same for “orange”. Important: use the same context words in both cases

The core idea: tables of counts as coordinates in a semantic space

	eat	fall	ripe	slice	peel	tree	throw	fruit	pie	bite	crab
apple	794	244	47	221	208	160	145	156	109	104	88
orange	265	22	25	62	220	64	74	111	4	4	8

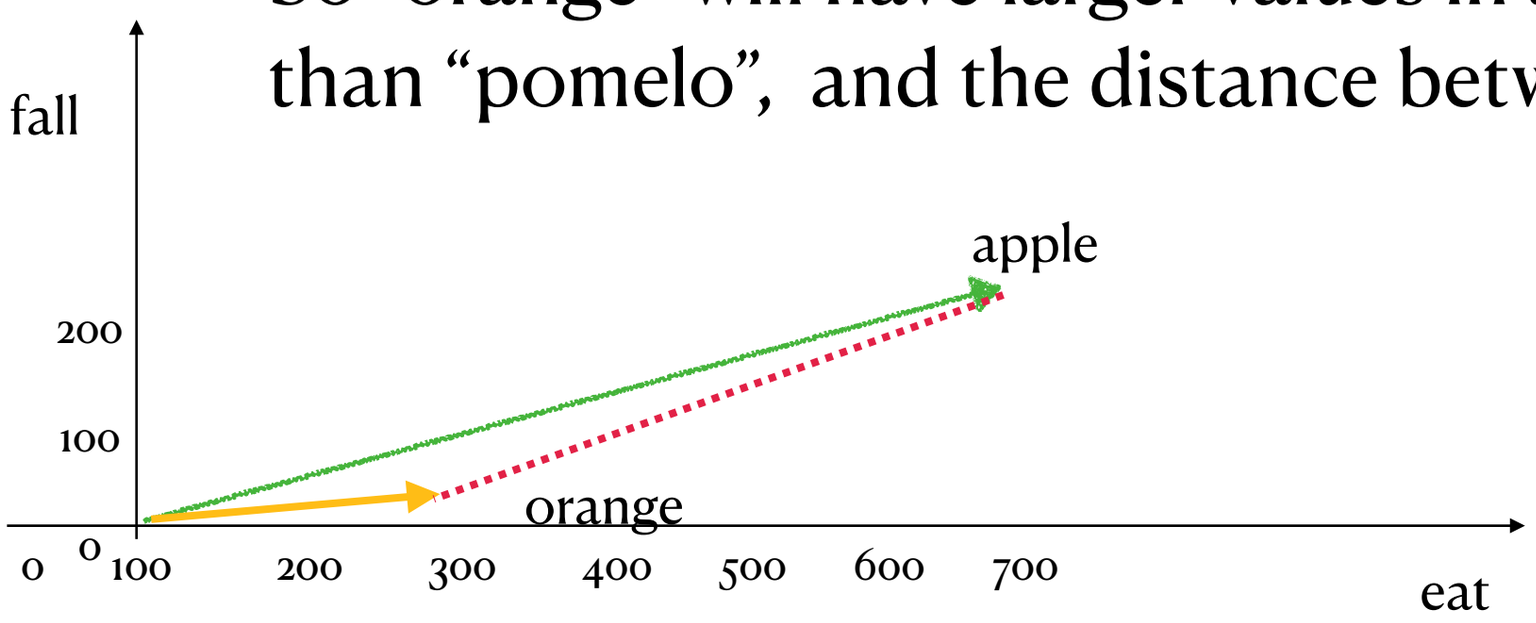


Similarity between two words as **proximity in a semantic space**

How do you measure the proximity in space of “apple” and “orange”?

Option 1: proximity = “walking distance”: Euclidean distance

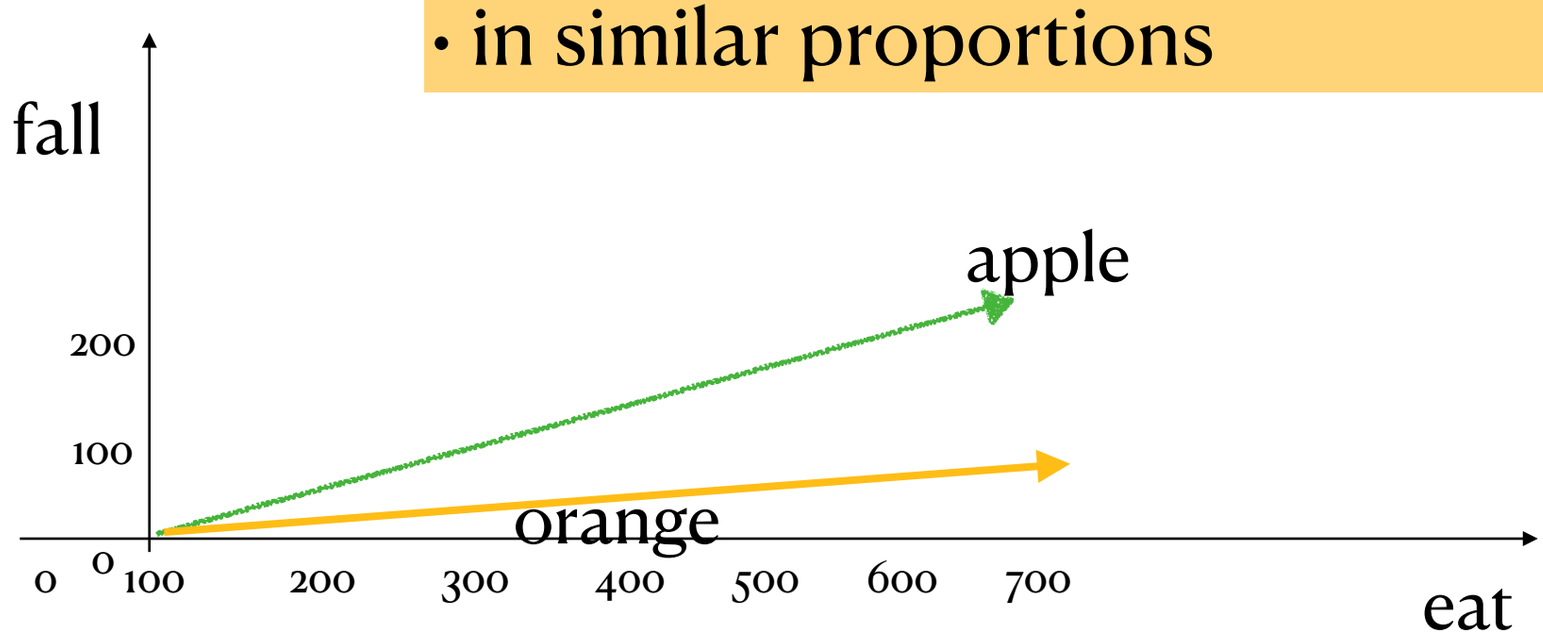
Problem: Some words are more frequent than others.
Think “orange” versus “pomelo”.
So “orange” will have larger values in all dimensions than “pomelo”, and the distance between them will be large.



How do you measure the proximity in space of “apple” and “orange”?

Option 2: **Cosine similarity**: make all vectors equally long first!

Two vectors are similar if they point in roughly the same direction:
- similar context words
- in similar proportions

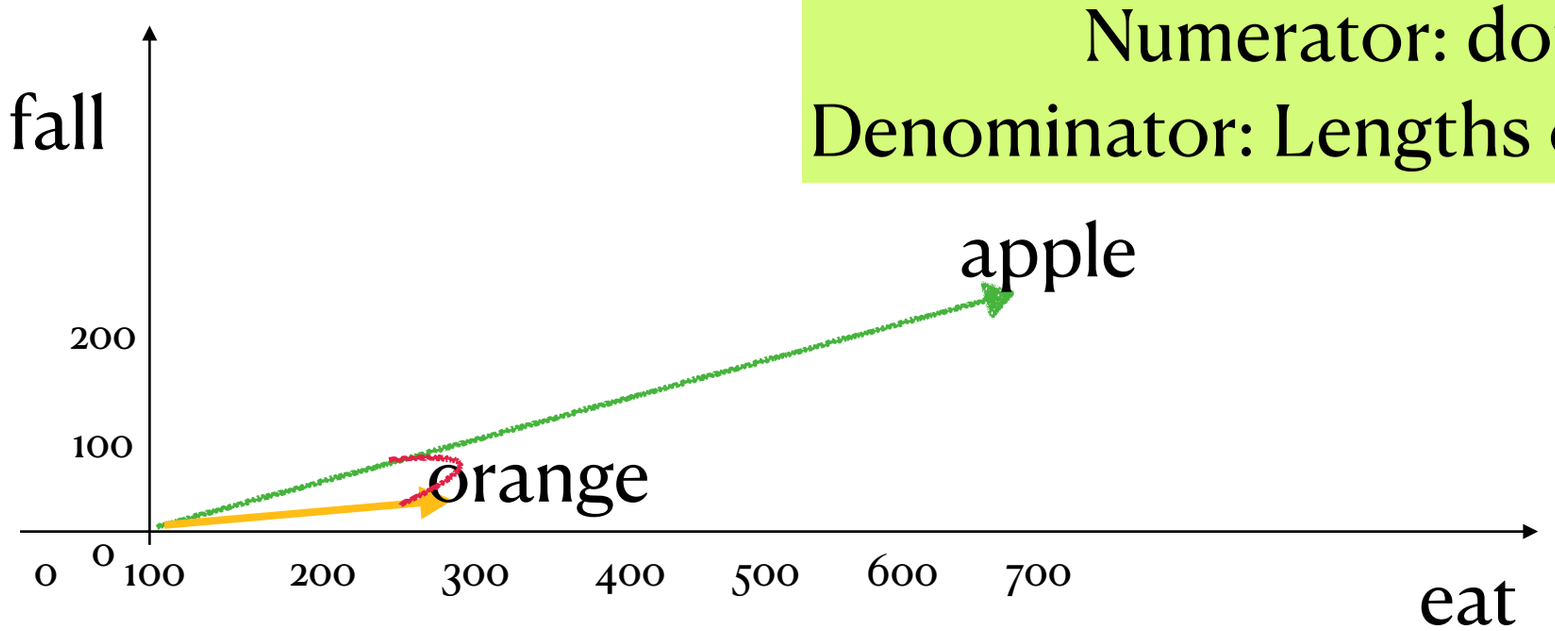


How do you measure the proximity in space of “apple” and “orange”?

Option 2: **Cosine similarity:** cosine of the angle between the vectors

$$\cos(\vec{p}, \vec{q}) = \frac{\sum_{i=1}^{n} p_i \cdot q_i}{\sqrt{\sum_{i=1}^{n} p_i^2}\sqrt{\sum_{i=1}^{n} q_i^2}}$$

Numerator: dot product
Denominator: Lengths of the two vectors



How do you measure the proximity in space of “apple” and “orange”?

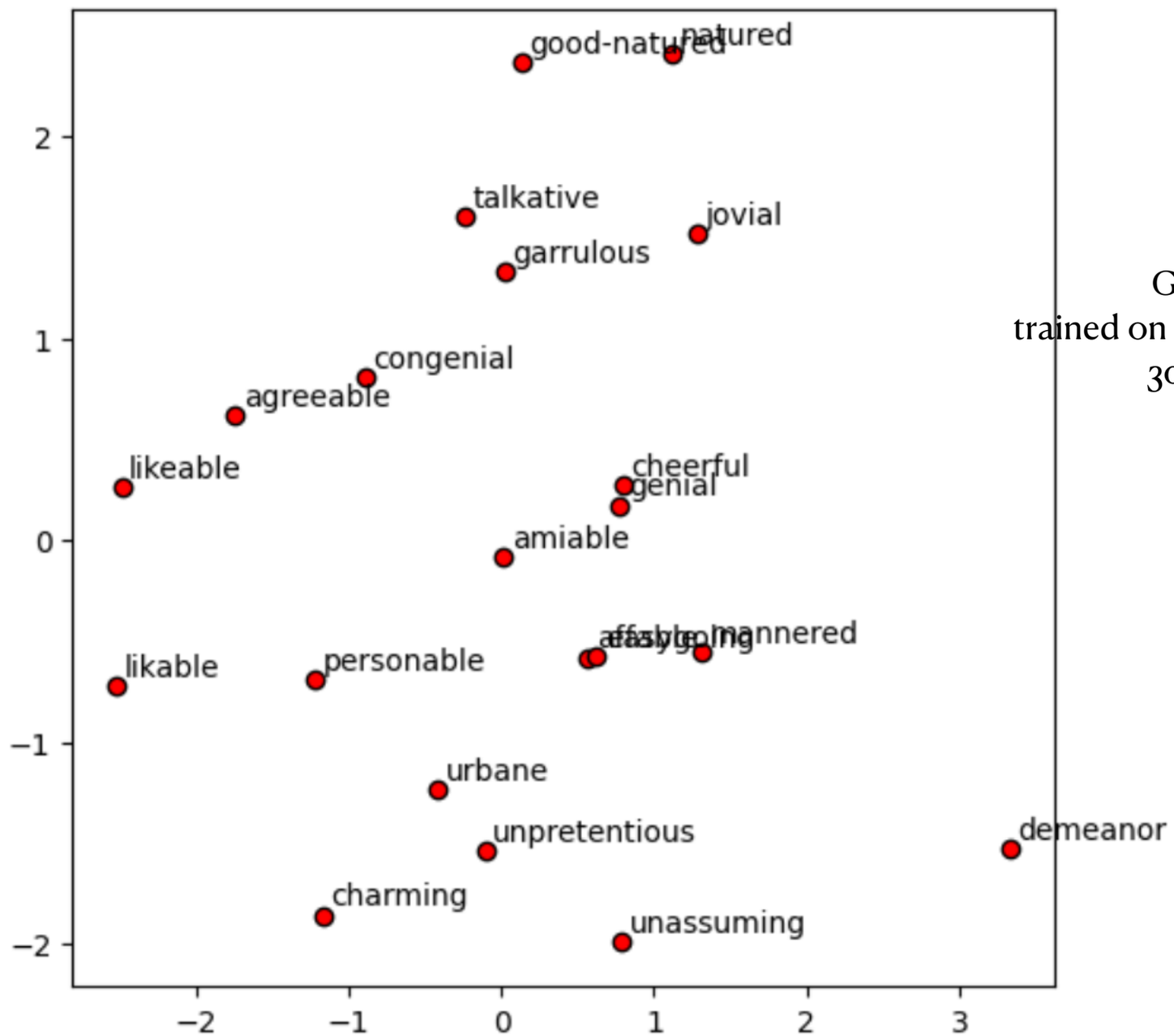
Cosine similarity example:

$$\cos(\vec{p}, \vec{q}) = \frac{\sum_{i=1}^{n} p_i \cdot q_i}{\sqrt{\sum_{i=1}^{n} p_i^2}\sqrt{\sum_{i=1}^{n} q_i^2}}$$

- p = (1,2,3), q = (2,1,4), then we get:
- Numerator: p1 q1 + p2 q2 + p3 q3 =
- 1*2 + 2 * 1 + 3 * 4 = 16
- Denominator: sqrt(1*1 + 2*2 + 3*3) * sqrt(2*2 + 1*1 + 4*4) = sqrt(14) * sqrt(21) = 3.74 * 4.58 = 17.15
- Cosine: 16 / 17.15 = 0.93

Vectors form a semantic space where similar meanings are close together

word: amiable

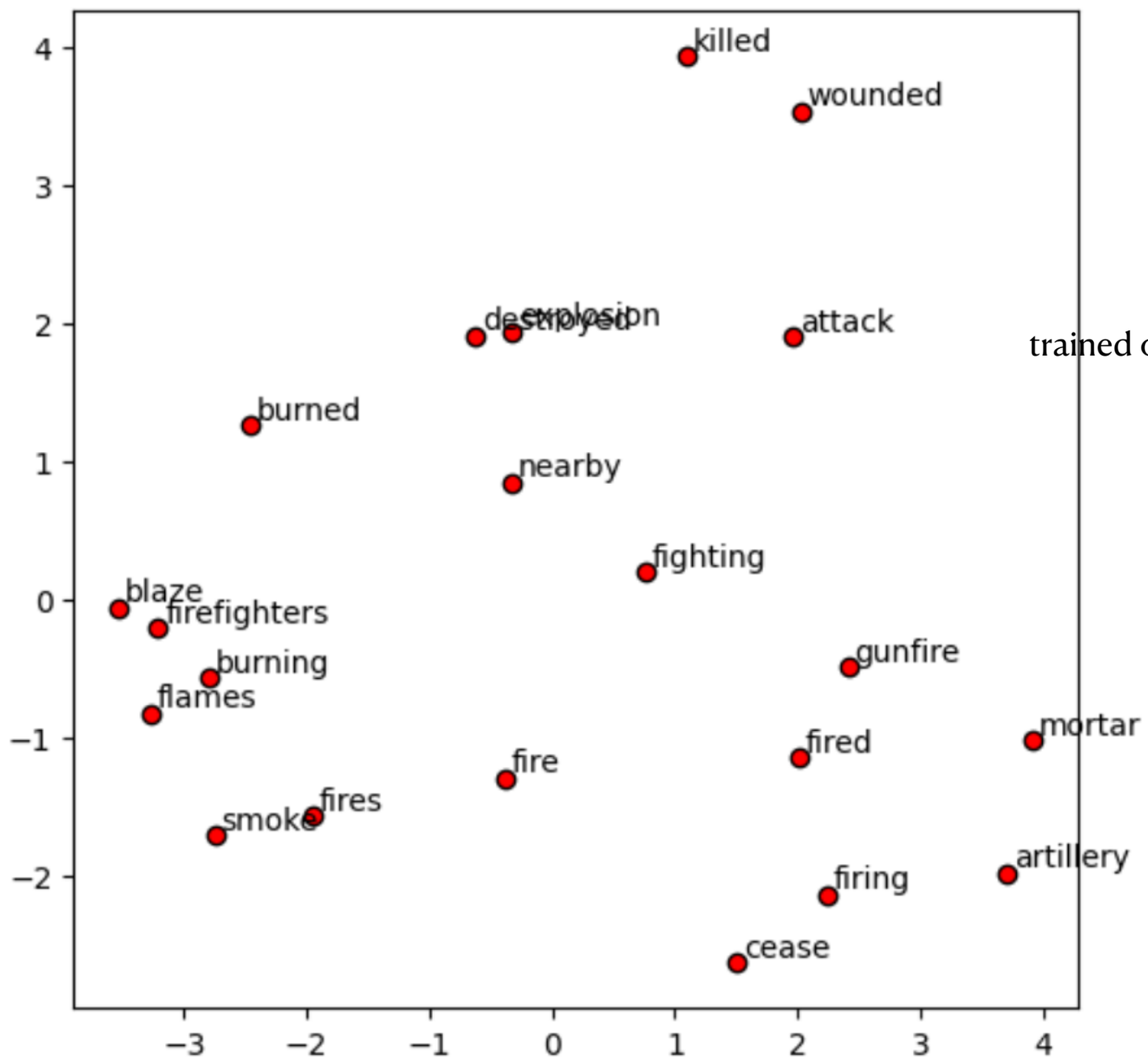


GloVE vectors,
trained on Wikipedia + Gigaword,
300 dimensions

Downprojection
to 2 dimensions: PCA

Vectors form a semantic space where similar meanings are close together

word: fire



GloVE vectors,
trained on Wikipedia + Gigaword,
300 dimensions

Downprojection
to 2 dimensions: PCA

From count-based models to prediction-based models

- Count-based models: count occurrences of context words, use to derive vectors
- Prediction-based models: predict which other words occur in the context, use to derive vectors
- The vectors can be used in the same way:
  - They form a “semantic space” where similar meanings are close together
  - Similarity typically measured using cosine

Key idea: Use unlabeled text as implicitly supervised data

- A word w near "apricot"
  acts as gold "correct answer" to the question
  "Is word w likely to show up near 'apricot'?"
- No need for hand-labeled supervision

Skip- Gram with Negative Sampling (SGNS)

The brown fox **jumps** over the lazy dog

<u>**SG**</u> NS: Skip- Gram Model

The brown fox **jumps** over the lazy dog.

Simple idea: from a word, predict its context words!
(A funny type of language model.)
Learn a vector that's good at that. Similar words should get similar vectors.

Word2Vec SkipGram: For words w, t: Is w a context word for t?

- For words w, t: Is w a word that has appeared in the context of t (in some given corpus)?
- Classifier: yes/no
- Say t = apricot, w = jam. What do you guess, yes or no?
- Say t = apricot, w = T-rex. What do you guess, yes or no?

What data do we use to train this classifier?

- For words w, t: Is w a context word for t?
- Positive data: corpus co-occurrences
  "Brush cake with strained **apricot jam** and place marzipan on top."
- Positive data: (apricot, jam), (apricot, and), (apricot, strained), (apricot, with)
- Where can we get negative data?

What data do we use to train this classifier?

- Use more negative than positive data by a factor of k (parameter of the model)
- Sample negative words w by weighted unigram

probability $p_\alpha(w) = \frac{count(w)^\alpha}{\sum_{w'} count(w')^\alpha}$

- typically, α = 0.75
- What does this do?

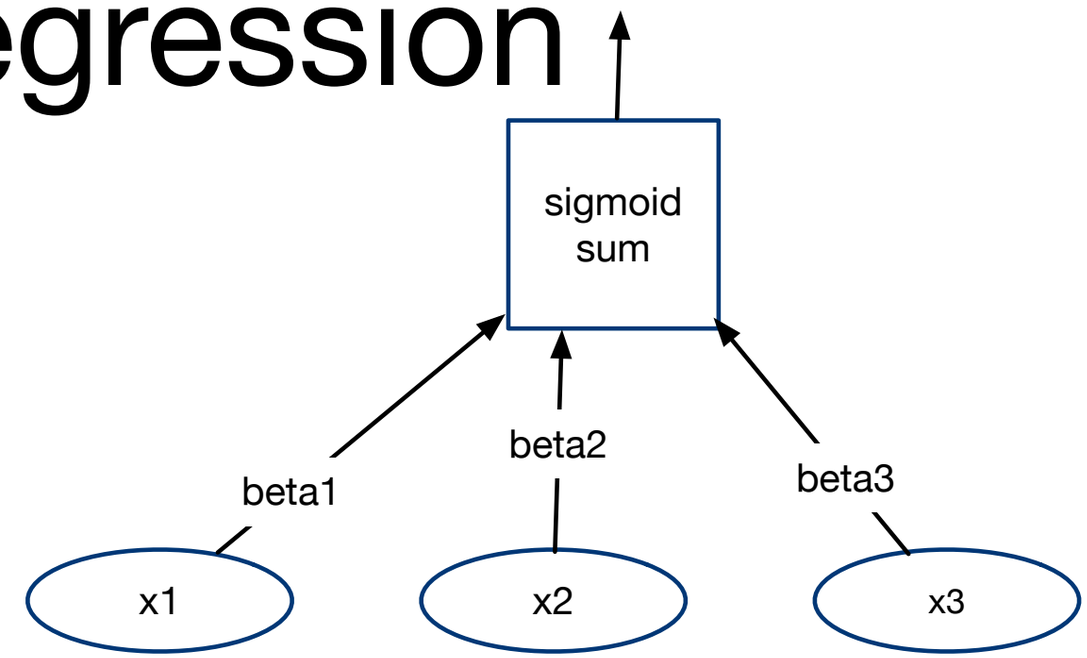
What data do we use to train this classifier?

- Setting alpha = 0.75 gives a bit of a boost to rare events. For example:

$$P_\alpha(0.99) = \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = 0.97$$

$$P_\alpha(0.01) = \frac{.01^{.75}}{.01^{.75} + .99^{.75}} = 0.03$$

Reminder: logistic regression



- Function notation:
  - $z = \beta \mathbf{x}$
  - $\hat{y} = \sigma(z) = \sigma(\beta \mathbf{x})$
- Parameters: β's
- Estimation: Loss function describing distance of ŷ from true y,
  Gradient descent for iteratively adjusting parameters

Word2Vec SkipGram: Compute word vectors that behave just like count-based vectors

- Learning a semantic space = learning the **parameters** of a model
- Logistic regression: Learn the weights β_i for the different inputs x_i
- Parameters to be learned here: The components of the vectors vec(t), vec(w)
- Like logistic regression: **classifier**: (t, w) cooccurring or not?
- Like logistic regression: **Loss function** to specify how "off" the prediction $\hat{y}$ is compared to y

Word2Vec SkipGram: For words w, t: Is w a context word for t?

- Compute word vectors that behave just like count-based vectors
- Every target word t has a vector vec(t)
- Every context word w has a vector vec(w)
  - (Let's say that target vectors and context vectors don't have to be the same)

Word2Vec SkipGram: For words w, t: Is w a context word for t?

- For w's that are **actual context words of t,** we want the **dot product** vec(t) * vec(w) to be **high**
  - Dot product: the numerator of cosine similarity

- For w's that we've **never seen as contexts** of t, we want the dot product vec(t) * vec(w) to be **low**

Word2Vec SkipGram: Compute word vectors that behave just like count-based vectors

- Estimate the probability that w is actually a seen context word for t:
  - P(+ | t, w)
- We predict that w is a context word for t if their similarity is high.
  Similarity: Dot product: $sim(t, w) = vec(t) \cdot vec(w)$
- $P(+ \mid t, w) = \sigma(\mathbf{tw}) = \dfrac{1}{1 + \exp(-\mathbf{wt})}$
- $P(- \mid t, w) = 1 - P(+ \mid t, w) = \sigma(-\mathbf{wt}) = \dfrac{1}{1 + \exp(\mathbf{wt})}$

Loss function says: P(+) should be high for positive datapoints, P(-) should be high for negative datapoints

- Target word t, positive context word w_{pos}, negative samples w_{neg1}, ...w_{negn}
- $$L(t, w_{pos}, w_{neg}*) = -\log\left(P(+ \mid t, w_{pos}) \prod_{i=1}^{n} P(- \mid t, w_{neg,i})\right)$$

$$= -\left(\log P(+ \mid t, w_{pos}) + \sum_{i=1}^{n} \log P(- \mid t, w_{neg,i})\right)$$

$$= -\left(\log P(+ \mid t, w_{pos}) + \sum_{i=1}^{n} \log(1 - P(+ \mid t, w_{neg,i}))\right)$$

$$= -\left(\log \sigma(w_{pos} t) + \sum_{i=1}^{n} \log \sigma(-w_{neg,i} t)\right)$$

Loss function says: P(+) should be high for positive datapoints, P(-) should be high for negative datapoints

- Target word t, positive context word w_{pos}, negative samples w_{neg1}, ...w_{negn}
- $$L(t, w_{pos}, w_{neg}*) = -\log\left(P(+ \mid t, w_{pos}) \prod_{i=1}^{n} P(- \mid t, w_{neg,i})\right)$$

$$= -\left(\log P(+ \mid t, w_{pos}) + \sum_{i=1}^{n} \log P(- \mid t, w_{neg,i})\right)$$

$$= -\left(\log P(+ \mid t, w_{pos}) + \sum_{i=1}^{n} \log(1 - P(+ \mid t, w_{neg,i}))\right)$$

$$= -\left(\log \sigma(w_{pos} t) + \sum_{i=1}^{n} \log \sigma(-w_{neg,i} t)\right)$$

Maximize dot product of t with actual context word, minimize dot product of t with negative samples

Word embeddings summary

- Count-based word vectors: Words that appear in similar textual contexts tend to be similar in meaning
- Word2vec SkipGram with negative sampling:
  Instead of counting context words in a corpus,
  predict, for pair (t, w),
  whether w appears as context item for t in the corpus
- Parameters of the model are word vectors
- Loss function: Vector similarity (dot prod.) high for target/context pairs, low for target/negsample pairs

Types of word embeddings

- Word2Vec with variants SkipGram, CBOW
- GLoVE
- FastText
- Precomputed embeddings available online
- Python package gensim offers access to precomputed Word2vec and GloVE vectors
  (Problem: gensim doesn't work with Python 3.14. I use a virtual environment with an older Python.)